ML21 Compare, explain and critique

Week 7, Your project | Session 21 of 24 | Machine Learning | New library: sklearn

Today's question. Does my model beat the baseline fairly, and can I defend each input?

Objective. By the end: you can compare candidate models on the same validation rows, explain why one input is excluded, and check a coding assistant's advice with tests.

> **WHERE THIS IS USED**
> When a model helps decide something about people, such as a loan, others ask why. The team must explain which inputs it used and defend them. Can you defend every input in your model?

Before this session: ML20 Build the project baseline. If you cannot explain the last example, use its error card with the instructor first.

Words for today

Word	What it means	Think of it like this
fair comparison	Same rows, same measure, same split for every model.	Every runner on the same track.
candidate model	A model you are considering using.	A player trying out for the team.
validation error	The average miss on validation rows.	The score from a trial run.
coding assistant	A tool that suggests code or fixes; its advice must be checked.	A friend who sounds sure and is sometimes wrong.
AI use log	A record of any assistance, advice taken or rejected, and tests run.	A signed note of who helped.

Block A. A fair comparison

Compare four options on the same 30 validation trips: the plain guess, distance only, three inputs, and three inputs plus tip. A small function keeps every comparison the same.

Predict. Predict which option has the lowest error, and whether you may use it. Write your guess on paper first.

Your guess, written before you run it

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.dummy import DummyRegressor
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error
fares = pd.read_csv("synthetic_fares.csv")
```

```
y = fares["fare_ugx"]

def validation_mae(columns, model):
    X = fares[columns]
    X_dev, X_test, y_dev, y_test = train_test_split(X, y, test_size=0.2, random_state=7)
    X_train, X_val, y_train, y_val = train_test_split(X_dev, y_dev, test_size=0.25,
        random_state=7)
    model.fit(X_train, y_train)
    return mean_absolute_error(y_val, model.predict(X_val))

base = validation_mae(["distance_km"], DummyRegressor(strategy="mean"))
distance_only = validation_mae(["distance_km"], LinearRegression())
three = validation_mae(["distance_km", "rush_hour", "raining"], LinearRegression())
with_tip = validation_mae(["distance_km", "rush_hour", "raining", "tip_ugx"],
    LinearRegression())
print(round(base), round(distance_only), round(three), round(with_tip))
```

Run it, then compare. The cell prints 4361, 2415, 2255 and 1317. The model with the tip has the lowest error, and it must not be used, because the tip is known only after the trip. Among the allowed models the three-input line wins: it misses by about UGX 2255, against 4361 for the plain guess. Every option used the same split, the same 30 validation trips and the same measure, which is what makes the comparison fair. The held-back set is untouched.

Investigate.

1. Why is the lowest error not the winner here?
2. What stays the same in a fair comparison?

Modify. Try distance and raining only, and print the validation error.

Block B. Check an assistant's advice

A function should multiply a price by a quantity but gives the wrong answer. A tutor shows an assistant's hint, or a tutor-written one, and you test it. Use only this practice function; your project stays separate. You already know functions and assert from session 6.

Predict. Predict what total(2000, 3) gives with the plus sign, and what it should give. Write your guess on paper first.

Your guess, written before you run it

```
def total(price, quantity):
    return price + quantity

buggy_result = total(2000, 3)
print(buggy_result)

def total(price, quantity):
    return price * quantity

assert total(2000, 3) == 6000
assert total(1000, 0) == 0
```

```
assert total(1000, 1) == 1000
```

Run it, then compare. The first print shows 2003, because plus adds the numbers. The required answer is 6000, and multiplying fixes it. The three asserts pass silently. The tests support this small requirement and do not prove the function is right for every input. The tutor's hint says to inspect the arithmetic operator and to try total(2000, 3) and total(1000, 0). A claim that every bug in a total function is fixed by changing + to * goes too far: a different bug would need a different repair.

Investigate.

1. What is useful in the hint and what is unjustified?
2. Why test with a quantity of 0?
3. What goes in AI_USE_LOG.md?

Modify. Add one more assert of your own and run the cell.

Make

Add model code, the validation error and a short decision to project.ipynb. Ask a partner to name your target and baseline from the notebook alone. Record any assistance in AI_USE_LOG.md or write No AI assistance used.

Save your work as ML21. Keep the worked example and put your own attempt below it. Write two sentences: what you changed and how you checked it.

Error card

You see	It means	Fix
The with-tip score looks like the best	A leakage column made the model look better.	Remove the column that is only known after the answer.
The comparison changes every time I run it	The split uses different random_state values or none at all.	Use the same split settings for every model.

Exit ticket

A model counts as better only on the same rows, with the same measure, and without leakage.

AI use follows AI_USE_POLICY.pdf. In weeks 1 to 6 no AI-generated solutions are used for exercises, labs or build tasks. Practise for 20 to 30 minutes between classes by repeating the Make task with one value changed. If you are stuck for 30 minutes, save the error and bring it to class. A file saved on your laptop has not been shared until you use the route in START_HERE.pdf.